ADVANCED PYTHON QUIZ

Introduction to Data Science

Name: _____ Roll No.: _____

Time: 50 minutes **Total Marks: 30**

Instructions: Choose one best answer for each MCQ. For coding questions, write executable Python 3 code. No Git questions are included. Do not use pandas.

SECTION A: PYTHON MCQs

10 questions × 1 mark = 10 marks

Q1. What is printed by this code?

```
a = [[1], [2]]
b = a.copy()
b[0].append(9)
b[1] = [7]
print(a)
```

A) [[1, 9], [2]]

B) [[1], [7]]

C) [[1, 9], [7]]

D) [[1], [2]]

Q2. What is the output?

```
nums = [0, 1, 2, 3, 4, 5, 6]
print(nums[5:1:-2])
```

A) [5, 3]

B) [5, 3, 1]

C) [1, 3, 5]

D) [6, 4, 2]

Q3. What does the list comprehension produce?

```
result = [(x, y) for x in range(3) for y in range(x)
        if (x + y) % 2 == 1]
print(result)
```

A) [(1, 0), (2, 1)]

B) [(1, 0), (2, 0)]

C) [(1, 0), (2, 0), (2, 1)]

D) [(0, 1), (1, 2)]

SECTION A: PYTHON MCQs (CONTINUED)

Select one best answer for each question.

Q4. Which three lines are printed, in order?

```
def add(x, cache=[]):
    cache.append(x)
    return cache

print(add(1))
print(add(2, []))
print(add(3))
```

A) [1] / [2] / [1, 3]

B) [1] / [1, 2] / [1, 2, 3]

C) [1] / [2] / [3]

D) [1] / [1, 2] / [3]

Q5. What is printed? Assume the dictionary is exactly as shown.

```
d = {"a": None}
print(d.get("a", 5), d.get("b", 5), d["a"])
```

A) None 5 None

B) 5 5 None

C) None None None

D) None 5 KeyError

Q6. Which output is produced by this exception-handling code?

```
try:
    int("4.2")
except ValueError:
    print("E")
else:
    print("S")
finally:
    print("F")
```

A) E, followed by F on the next line

B) S, followed by F on the next line

C) E, then S, then F on separate lines

D) A ValueError traceback, followed by F

SECTION A: PYTHON MCQs (CONTINUED)

Select one best answer for each question.

Q7. What is the value printed for `d`?

```
pairs = [("a", 1), ("b", 2), ("a", 3)]
d = {k: v for k, v in pairs}
print(d)
```

A) {'a': 3, 'b': 2}

B) {'a': 1, 'b': 2}

C) {'a': 1, 'b': 2, 'a': 3}

D) {'b': 2, 'a': 3}

Q8. What is printed?

```
items = ["x", "y", "z", "w"]
print([(i, v) for i, v in enumerate(items, start=1)
    if i % 2 == 0])
```

A) [(2, 'y'), (4, 'w')]

B) [(1, 'x'), (3, 'z')]

C) ['y', 'w']

D) [(0, 'x'), (2, 'z')]

Q9. What are the three outputs, in order?

```
g = (x * x for x in range(4))
print(next(g))
print(list(g))
print(list(g))
```

A) 0 / [1, 4, 9] / []

B) 0 / [0, 1, 4, 9] / [0, 1, 4, 9]

C) 1 / [4, 9] / []

D) 0 / [1, 4, 9] / [1, 4, 9]

Q10. What is printed? Consider how `sorted` handles items with equal sort keys.

```
words = ["pear", "fig", "plum", "kiwi"]
print(sorted(words, key=len))
```

A) ['fig', 'pear', 'plum', 'kiwi']

B) ['fig', 'kiwi', 'pear', 'plum']

C) ['pear', 'plum', 'kiwi', 'fig']

D) ['fig', 'pear', 'kiwi', 'plum']

SECTION B: PYTHON CODING CHALLENGES

5 questions × 4 marks = 20 marks. Use Python 3 and built-in data structures only; do not use pandas.

Shared dataset - use this data for all coding questions:

```
students = [
    {"name": "Ali", "allowance": 10000, "expenses": [1500, 2200, 1800, 3000], "category": "Regular"},
    {"name": "Sara", "allowance": 8000, "expenses": [2500, 3100, 2800], "category": "Scholarship"},
    {"name": "Usman", "allowance": 12000, "expenses": [1000, 1200], "category": "Regular"},
    {"name": "Ayesha", "allowance": 6000, "expenses": [2000, 2500, 1800], "category": "Scholarship"},
    {"name": "Zain", "allowance": 15000, "expenses": [4000, 5000, 4500], "category": "Regular"},
]
```

Q1. Build a student summary. (4 marks)

Write code that loops through `students` and builds a new list named `summary`. Each element must be a dictionary containing `name`, `total_spent`, `balance`, and `high_expenditure`. Set `high_expenditure` to `True` only when total spending is strictly greater than 80% of the allowance. Do not modify any original student dictionary. Print `summary`.

Q2. Calculate scholarship bonuses. (4 marks)

Create a dictionary named `scholarship_bonuses`. Include only students whose category is `"Scholarship"`. Each value must be 10% of the student's remaining balance (`allowance - sum(expenses)`). If the remaining balance is zero or negative, use a bonus of `0`. Round every bonus to 2 decimal places. Print the dictionary.

Q3. Rank the highest-spending students. (4 marks)

Write a function `rank_students(students, top_n=3)` that returns a new list of `(name, total_spent)` tuples sorted from highest total spending to lowest. If two students have the same total, sort their names alphabetically. The function must not reorder or modify `students`. Demonstrate it by printing `rank_students(students)`.

Q4. Record a purchase safely. (4 marks)

Implement `record_purchase(students, name, amount)`. If a student with the exact name exists and `amount` is a non-negative `int` or `float`, append it to that student's `expenses` and return `True`. Reject booleans as amounts. Return `False` if the name is not found or the amount is invalid. On failure, leave `students` unchanged.

Q5. Create a category-level report. (4 marks)

Write `category_report(students)` that returns a tuple `(report, highest_spending_category)`. `report` must map each category to a dictionary containing `student_count`, `combined_expenses`, and `over_80_percent_count` (the number of students in that category whose total spending is strictly greater than 80% of allowance). Choose the category with the greatest `combined_expenses`; if tied, choose the alphabetically first category. Do not use pandas.